

Divergences

- Divergences occur when the simulated Hamiltonian trajectory departs from the true trajectory (leapfrog isn't accurate)
- Sometimes it helps to reduce the step size or increase acceptance rate (`adapt_delta`)
- Reparameterization can be very helpful!
↳ sometimes indicates poor model fit.

Example: Neal's Funnel

- Exemplifies the difficulties of sampling from some hierarchical models
- The probability contours are shaped like ten-dimensional funnels.

Example: Neal's Funnel

```
1 parameters {  
2   real y;  
3   vector[9] x;  
4 }  
5 model {  
6   y ~ normal(0, 3);  
7   x ~ normal(0, exp(y/2));  
8 }
```

Neal's Funnel

Running MCMC with 4 sequential chains...

```
Chain 1 Iteration:    1 / 2000 [  0%] (Warmup)
Chain 1 Iteration:   100 / 2000 [  5%] (Warmup)
Chain 1 Iteration:   200 / 2000 [ 10%] (Warmup)
Chain 1 Iteration:   300 / 2000 [ 15%] (Warmup)
Chain 1 Iteration:   400 / 2000 [ 20%] (Warmup)
Chain 1 Iteration:   500 / 2000 [ 25%] (Warmup)
Chain 1 Iteration:   600 / 2000 [ 30%] (Warmup)
Chain 1 Iteration:   700 / 2000 [ 35%] (Warmup)
Chain 1 Iteration:   800 / 2000 [ 40%] (Warmup)
Chain 1 Iteration:   900 / 2000 [ 45%] (Warmup)
Chain 1 Iteration:  1000 / 2000 [ 50%] (Warmup)
Chain 1 Iteration: 1001 / 2000 [ 50%] (Sampling)
```

variable	mean	median	sd	mad	q5	q95	rhat	ess_bulk	ess_tail
lp__	-4.81	-3.70	11.67	11.11	-26.05	14.32	1.09	49	12
y	0.01	-0.17	2.54	2.40	-4.53	4.56	1.09	47	12
x[1]	-0.04	0.02	4.67	0.87	-4.63	4.47	1.04	3867	463
x[2]	0.09	0.03	4.52	0.83	-4.13	4.32	1.05	3036	456
x[3]	-0.07	-0.01	4.69	0.81	-4.30	4.06	1.05	2876	403
x[4]	-0.12	-0.03	4.42	0.77	-4.37	4.12	1.05	3190	454
x[5]	-0.15	0.00	5.09	0.82	-4.84	4.28	1.05	2937	436

x[6]	-0.08	0.00	4.47	0.79	-4.68	4.37	1.05	1964	290
x[7]	-0.22	0.00	4.74	0.86	-4.14	3.65	1.05	2723	339
x[8]	0.04	-0.01	4.27	0.84	-4.22	4.31	1.05	4078	420

showing 10 of 11 rows (change via 'max_rows' argument or
'cmdstanr_max_rows' option)

Neal's Funnel

```
1 parameters {  
2   real y_raw;  
3   vector[9] x_raw;  
4 }  
5 transformed parameters {  
6   real y;  
7   vector[9] x;  
8  
9   y = 3.0 * y_raw;  
10  x = exp(y/2) * x_raw;  
11 }  
12 model {  
13   y_raw ~ std_normal(); // implies y ~ normal(0, 3)  
14   x_raw ~ std_normal(); // implies x ~ normal(0, exp(y/2))  
15 }
```

Neal's Funnel

Running MCMC with 4 sequential chains...

Chain 1 finished in 0.1 seconds.

Chain 2 finished in 0.1 seconds.

Chain 3 finished in 0.1 seconds.

Chain 4 finished in 0.1 seconds.

All 4 chains finished successfully.

Mean chain execution time: 0.1 seconds.

Total execution time: 0.5 seconds.

Running MCMC with 4 sequential chains...

Chain 1 finished in 0.0 seconds.

Chain 2 finished in 0.0 seconds.

Chain 3 finished in 0.0 seconds.

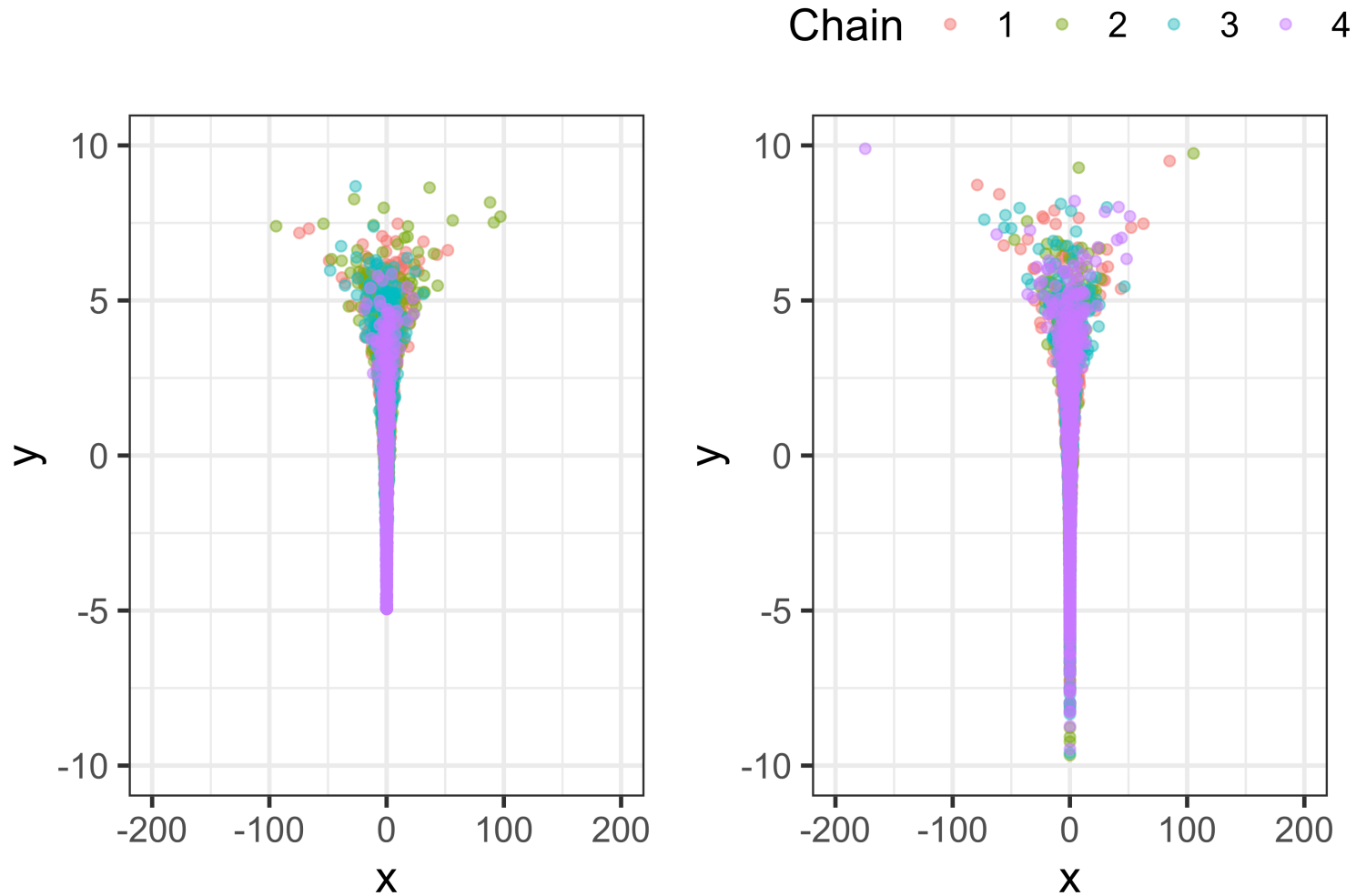
Chain 4 finished in 0.0 seconds.

All 4 chains finished successfully.

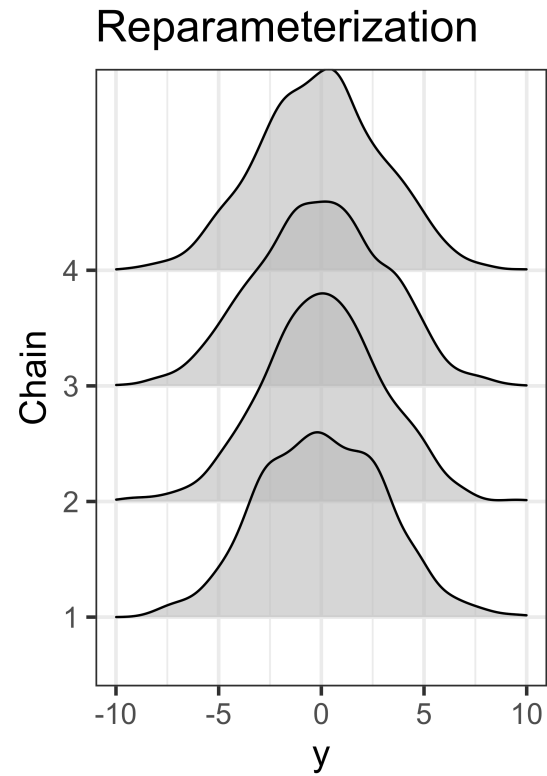
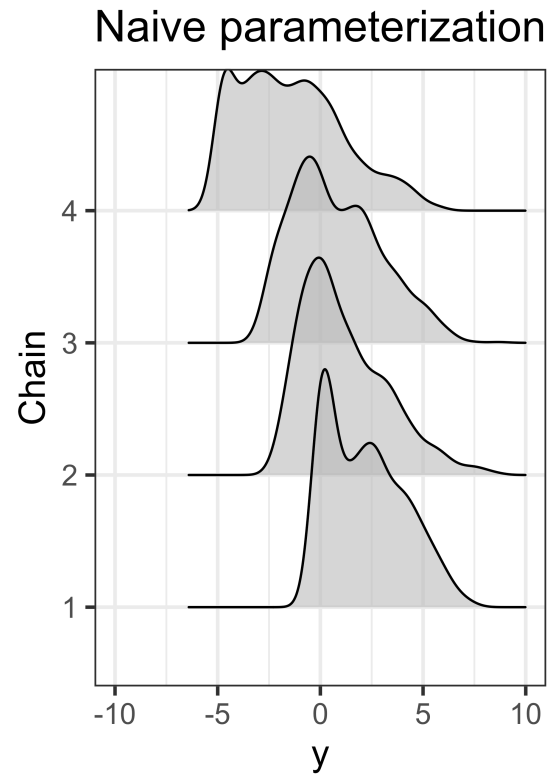
Mean chain execution time: 0.0 seconds.

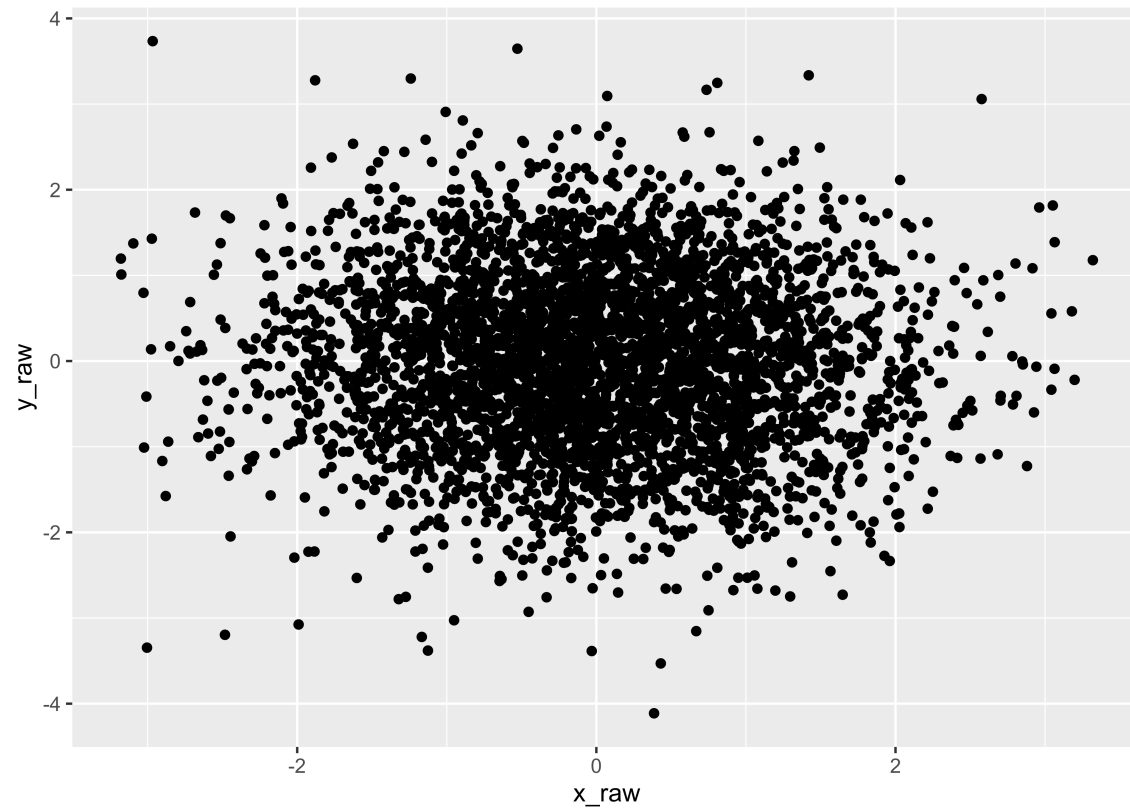
Total execution time: 0.5 seconds.

Neal's Funnel



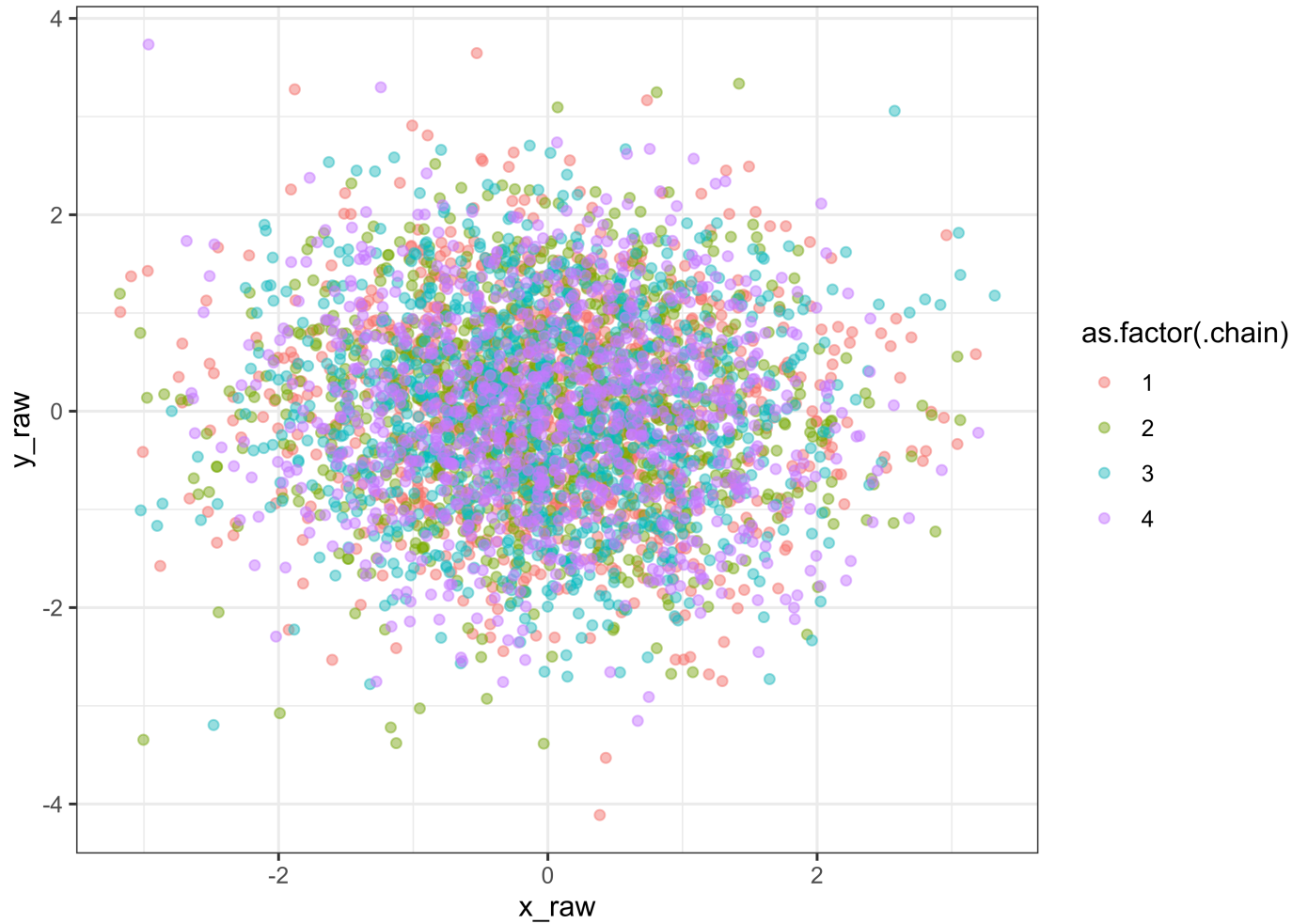
Neal's Funnel





Neal's Funnel

Sampling in the reparameterized space is *much* easier!!



Gibbs

HMC is a good default, but doesn't work with discrete parameters.

$$P(\theta | y) = P(\underline{\theta}_1, \underline{\theta}_2, \dots, \underline{\theta}_d | y)$$

for (i in 1:niters) {

 for (j in 1 to d) {

$\theta_j^{(i)}$

$\sim P(\underline{\theta}_j | \theta_{-j}, y)$

current value
of all other
params

"complete
conditional"

 }

}

The Gibbs Sampler

- Suppose that the parameter vector θ can be divided into d subvectors.
- For sample $s = 1, \dots, S$:
 - For $j \in 1, \dots, d$
 - Draw a value from the conditional distribution of θ_j given all the other parameters,

$$\theta_j^s \sim p(\theta_j | \theta_{-j}^{(s-1)}, y),$$

where θ_{-j} consists of updated parameters for all parameters preceding j and the previous iteration's values for all succeeding parameters,

$$\theta_{-j}^{s-1} = \left(\theta_1^s, \dots, \theta_{s-1}^t, \theta_{j-1}^{s-1}, \dots, \theta_d^{s-1} \right).$$

The Gibbs Sampler

- To identify the full conditional distributions:

1. Start with the full posterior
2. For each parameter, θ_j , remove all multiplicative factors which constant in θ_j
3. Identify the kernel of the resultant conditional density (or otherwise devise a strategy for sampling)

$$\text{e.g. } (\theta_a, \theta_b) \sim \mathcal{N}_2(0, \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix})$$

$$\theta_a | \theta_b \sim \mathcal{N}(\rho \theta_b, 1 - \rho^2)$$

$$\theta_b | \theta_a \sim \mathcal{N}(\rho \theta_a, 1 - \rho^2)$$

Gibbs Sampling the Normal Model

$$y_i \stackrel{iid}{\sim} \mathcal{N}(\mu, \sigma^2)$$

$$\mu \sim \mathcal{N}(\mu_0, \frac{\sigma^2}{\kappa_0})$$

$$\sigma^2 \sim 1/\sigma^2$$

$$P(\mu, \sigma^2 | y_1, \dots, y_n) \propto \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(y_i - \mu)^2}{2\sigma^2}}$$

$$1. \quad P(\mu | \sigma^2, y_1, \dots, y_n) \sim \mathcal{N}(w\bar{y} + (1-w)\mu_0, \frac{\sigma^2}{n+\kappa_0})$$

$$2. \quad P(\sigma^2 | \mu, y_1, \dots, y_n) \sim \text{Inv-Gamma}(\cdot, \cdot)$$

The Gibbs Sampler

- Assume $p(\mu) \sim N(\mu_0, \tau^2)$ and $\frac{1}{\sigma^2} \sim \text{Gamma}(a, b)$
- $p(\mu \mid \sigma^2, y)$ is a normal distribution (conjugacy)
 - Sample $\mu^{(s)} \sim \text{Normal}(\mu_n, \tau_n^2)$
 - $\tau_n^2 = \frac{\sigma^{2(s-1)}}{n + \kappa_0}$
- $p(\frac{1}{\sigma^2} \mid \mu, y)$ is a gamma distribution (conjugacy)
 - Sample
$$\left(\frac{1}{\sigma^2}\right)^{(s)} \sim \text{Gamma}\left(a + n/2, b + \frac{\sum (d_i^{(s)})^2}{2} + \frac{\kappa_0}{2} (\mu^{(s)} - \mu_0)^2\right)$$
 - $d_i^{(s)} = (y_i - \mu^{(s)})$

Gibbs Sampler

```
1 gibbs <- function(mu_cur, prec_cur, burnin, maxit)
2
3   nsamp <- maxit + burnin
4   mu_samps <- numeric(nsamp)
5   prec_samps <- numeric(nsamp)
6   mu_samps[1] <- mu_cur
7   prec_samps[1] <- prec_cur
8
9   for(i in 2:nsamp) {
10     ## Sample mu given current precision
11     mu_cur <- rnorm(1, mun, sqrt(1/prec_cur * 1/(k0 + n)))
12
13     ## Sample precision given current mu
14     prec_cur <- rgamma(1, n/2+1, sum((y - mu_cur)^2)/2 + k0/2*(mu_cur - mu0)^2)
15
16     mu_samps[i] <- mu_cur
17     prec_samps[i] <- prec_cur
18   }
```

Gibbs Sampler Code

```
1  ## ... cont
2  ## Chop off the first part of the chain -- this reduces dependence on t
3  if(burnin == 0) {
4    list(mu_samps = mu_samps, theta_samps=theta_samps)
5  }
6  else {
7    list(mu_samps=mu_samps[-(1:burnin)], , prec_samps=prec_samp[-(1:burni
8  }
9  }
```

The Gibbs Sampler

Animations

Gibbs Sampler: Normal Hierarchical Model

Gibbs Sampler

Gibbs: a special case of MH

(with 100% acceptance)

Target $P(\theta_1, \theta_2 | y)$

Complete conditional

Updating parameter 1: $J(\theta_1^* | (\theta_1, \theta_2)) = P(\theta_1^* | \theta_2, y)$

$$\text{Acc. Prob.: } \frac{P(\theta_1^*, \theta_2 | y) J(\theta_1 | \theta_1^*, \theta_2)}{P(\theta_1, \theta_2 | y) J(\theta_1^* | \theta_1, \theta_2)}$$

$$= \frac{P(\theta_1^* | \theta_2, y) P(\theta_2 | y) P(\theta_1 | \theta_2, y)}{P(\theta_1 | \theta_2, y) P(\theta_2 | y) P(\theta_1^* | \theta_2, y)} = 1$$

2/25

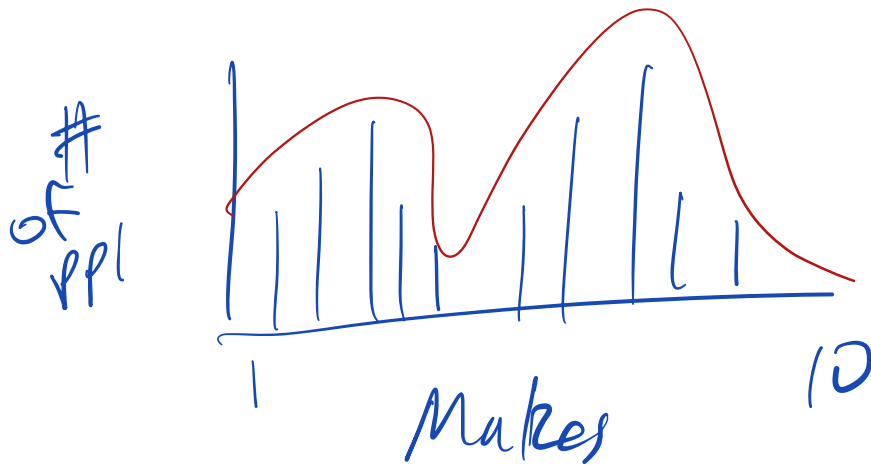
- HW 4 due (Read BDA on HMC)
- Final Project

Gibbs Sampler: Finite Mixture Models

Reminder: mixture of binomials example

```
1 set.seed(123)
2 n <- 25
3 phi <- 0.4
4 size <- 10
5 z <- ifelse(rbinom(100, 1, phi), "Experience", "No Experience")
6 y <- rbinom(n, size=size, prob=ifelse(z=="Experience", 0.7, 0.3))
```

Assume Don't observe z .



$$z_i \sim \text{Bern}(\phi)$$

$$y_i \sim \begin{cases} \text{Bin}(10, \theta_1) & z=1 \\ \text{Bin}(10, \theta_0) & z=0 \end{cases}$$

$$P(\theta_1, \theta_0, \phi | \dots)$$

$$P(\theta_0, \theta_1, \phi | \dots) = \int P(\theta_1, \theta_0, \phi, z | \dots) dz$$

$$L(\phi, \theta_0, \theta_1, z) \propto$$

$$\phi^{\sum z_i} (1-\phi)^{n-\sum z_i} \theta_1^{\sum z_i y_i} (1-\theta_1)^{\sum z_i (1-y_i)} \theta_0^{\sum (1-z_i) y_i} (1-\theta_0)^{\sum (1-z_i) (1-y_i)}$$

Inference via Gibbs

1. $P(\phi | \theta_0, \theta_1, \vec{z}, y's) \sim \text{Beta}(\sum z_i + \alpha_\phi, \sum (1-z_i) + \beta_\phi)$
2. $P(\theta_1 | \theta_0, \phi, z's, y's) \sim \text{Beta}(\sum z_i y_i + \alpha_{\theta_1}, \sum z_i (1-y_i) + \beta_{\theta_1})$
3. $P(\theta_0 | \dots) \sim \text{Beta}$
4. $P(z | \theta_0, \theta_1, \phi) \sim ??$

$$Z_i = 1: \phi \theta_1^{y_i} (1 - \theta_1)^{(1 - y_i)}$$

$$Z_i = 0: \theta_0^{y_i} (1 - \theta_0)^{(1 - y_i)} (1 - \phi)$$

$$Z_i | \dots \sim \text{Bern} \left(\frac{\theta_1^{y_i} (1 - \theta_1)^{(1 - y_i)} \phi}{\theta_1^{y_i} (1 - \theta_1)^{(1 - y_i)} \phi + \theta_0^{y_i} (1 - \theta_0)^{(1 - y_i)} (1 - \phi)} \right)$$

Star way.

$$P(\underline{\theta}_0, \underline{\theta}_1, \underline{\phi} | y's) \propto$$

$$\prod_{i=1}^n \left[\phi \binom{10}{y_i} \theta_1^{y_i} (1 - \theta_1)^{(1 - y_i)} + (1 - \phi) \binom{10}{y_i} \theta_0^{y_i} (1 - \theta_0)^{(1 - y_i)} \right]$$

Gibbs Sampler: Binomial Mixture

```
1 ## Initialize all
2 run_binomial_gibbs <- function() {
3   experience_vec <- rbinom(n, 1, 0.5)
4   theta_cur <- c(0.1, 0.9)
5   phi_cur <- 0.5
6   nsamps <- 10000
7
8   theta_samps <- matrix(0, nrow=nsamps, ncol=2)
9   colnames(theta_samps) <- c("theta1", "theta2")
10  phi_samps <- numeric(nsamps)
11  experience_samps <- matrix(0, nrow=nsamps, ncol=n)
12
13  for(i in 1:nsamps) {
14
15    ## Update experience indicators
16    experience_prob <- (phi_cur*dbinom(y, 10, prob=theta_cur[2])) / ((1-phi_cur)*dbinom(y, 10, prob=theta_cur[1]) + phi_cur*dbinom(y, 10, prob=theta_cur[2]))
17
18    ## Z | ... ~ Bern( p )
19
20    experience_vec <- 1*(runif(n) < experience_prob)
21    ## experience_vec <- rbern(experience_prob) ## equiv
22
23    ## Update mixture weights
24    phi_cur <- rbeta(1, sum(experience_vec)+1, n-sum(experience_vec)+1)
25    phi_samps[i] <- phi_cur
```

Stan version

```
1 library(cmdstanr)
2 library(tidyverse)
3 library(tidybayes)
4
5 sm <- cmdstan_model("binomial_mix.stan")
6 data_list=list(y=y, N=length(y), K=2)
7 stan_res <- sm$sample(data=data_list, iter_warmup=10000, iter_sampling=20000)
```

Running MCMC with 4 sequential chains...

```
Chain 1 Iteration:      1 / 30000 [  0%] (Warmup)
Chain 1 Iteration:    1000 / 30000 [  3%] (Warmup)
Chain 1 Iteration:    2000 / 30000 [  6%] (Warmup)
Chain 1 Iteration:    3000 / 30000 [ 10%] (Warmup)
Chain 1 Iteration:    4000 / 30000 [ 13%] (Warmup)
Chain 1 Iteration:    5000 / 30000 [ 16%] (Warmup)
Chain 1 Iteration:    6000 / 30000 [ 20%] (Warmup)
Chain 1 Iteration:    7000 / 30000 [ 23%] (Warmup)
Chain 1 Iteration:    8000 / 30000 [ 26%] (Warmup)
Chain 1 Iteration:    9000 / 30000 [ 30%] (Warmup)
Chain 1 Iteration:   10000 / 30000 [ 33%] (Warmup)
Chain 1 Iteration:   10001 / 30000 [ 33%] (Sampling)
```



```
1 print(stan_res)
```

variable	mean	median	sd	mad	q5	q95	rhat	ess_bulk	ess_tail
lp__	-60.07	-59.71	1.38	1.09	-62.74	-58.61	1.00	24178	29084
prop[1]	0.41	0.41	0.11	0.12	0.23	0.60	1.00	37135	31836
prop[2]	0.59	0.59	0.11	0.12	0.40	0.77	1.00	37135	31836
lambda[1]	0.63	0.52	0.45	0.38	0.11	1.50	1.00	26714	25635
lambda[2]	1.37	1.15	0.97	0.84	0.25	3.27	1.00	26004	24803
theta[1]	0.31	0.31	0.03	0.03	0.26	0.37	1.00	72472	41281
theta[2]	0.69	0.69	0.03	0.03	0.63	0.74	1.00	72472	41281

```
1 stan_res %>% spread_draws(theta[K]) %>%  
2   ggplot() + geom_line(aes(x=.iteration, y=theta, col=as.factor(.chain))) +  
3   facet_wrap(~K)
```

Quantile intervals

```
1 quantile(gibbs_results[[1]][, "theta1"], c(0.025, 0.5, .975))
```

2.5%	50%	97.5%
0.2399146	0.3738561	0.7823448

```
1 quantile(gibbs_results[[1]][, "theta2"], c(0.025, 0.5, .975))
```

2.5%	50%	97.5%
0.3040817	0.7194903	0.8339195

```
1 quantile(gibbs_results[[1]][, "phi_samps"], c(0.025, 0.5, .975))
```

2.5%	50%	97.5%
0.2355142	0.5119565	0.7879399

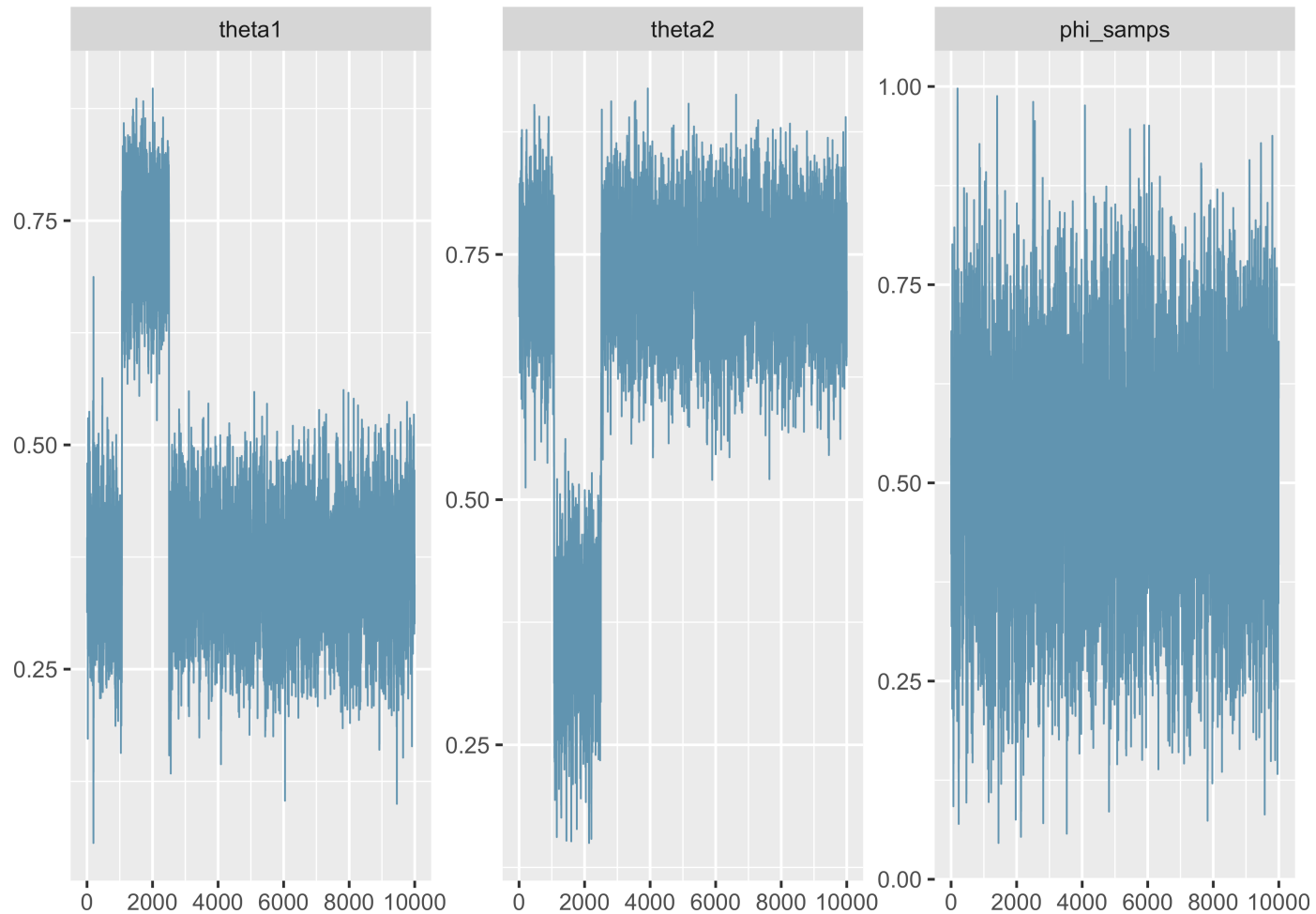
Rhat

```
1 rstan::Rhat(cbind(gibbs_results[[1]][, "theta1"], gibbs_results[[2]][, "the  
[1] 1.175579
```

```
1 rstan::Rhat(cbind(gibbs_results[[1]][, "theta2"],gibbs_results[[2]][, "thet  
[1] 1.174507
```

Traceplots

```
1 bayesplot::mcmc_trace(gibbs_results[[1]])
```



Correcting label switching

```
1 quantile(gibbs_results[[1]][, 1], c(0.05, .5, 0.95))
```

5%	50%	95%
0.2627420	0.3738561	0.7531172

```
1 quantile(apply(gibbs_results[[1]][, 1:2], 1, max), c(0.05, 0.5, 0.95))
```

5%	50%	95%
0.6328924	0.7315352	0.8209664

```
1 quantile(apply(gibbs_results[[1]][, 1:2], 1, min), c(0.05, 0.5, 0.95))
```

5%	50%	95%
0.2570862	0.3606355	0.4630786

The model is likelihood
non-identifiable because

$\Theta_1 > \Theta_0$ and $\Theta_0 > \Theta_1$
can give equally good
model fits.

Factor Model

$$\Sigma_{p \times p} = F F'_{p \times k \quad k \times p}$$

$$= \tilde{F} \tilde{F}'$$

where $\tilde{F} = F R$

orthogonal
matrix.

$$Y \sim N(0, \Sigma)$$

$$FF' = \begin{bmatrix} F_1 R^T F_1' & F_1 R^T F_2' \\ F_2 R^T F_1' & F_2 R^T F_2' \end{bmatrix}$$